TorchTPU · Gemma 2B greedy decoding on TPU v5e-1: Boosting inference to 21,645 tok/s

Rubens Zimbres
Google Developer Expert in Cloud and AI
rubenszmm@gmail.com

October 2026

DOI: 10.5281/zenodo.23840458

Abstract

TorchTPU is a PyTorch backend that runs PyTorch programs on Google TPUs through a native `tpu` device. This report measures greedy text generation with Gemma 2B on one TPU v5e chip using the public release `torch_tpu` 0.2.0. No serving engine exists for TorchTPU, so the model, the key and value cache, the decoding loop, and a decode attention kernel in Pallas were written for this study. The workload repeats a previous vLLM experiment by the author on the same chip class: 512 sequences and 128 generated tokens per sequence. Four experiments add one change at a time. Compiling the decode step with `torch.compile` raises throughput at batch 128 by a factor of 5.1, from 2,095 to 10,624 tokens per second. At batch 512, the Pallas kernel registered as a PyTorch operation adds 5.6 percent over compiler generated attention, and running eight decode steps inside one compiled program with the `scan` operator adds 21.1 percent. With the real weights the final configuration reaches 21,645.8 tokens per second, with a batch time to first token of 222 milliseconds, an inter token latency of 22.1 milliseconds, and identical tokens across repeated runs. The vLLM experiment had reached 19,511 tokens per second; the two systems differ in scheduling and cache design, which the report lists. An estimate of the arithmetic places the final configuration at about 55 percent of the chip's peak bfloat16 compute. The report explains each component, how each result was verified, and the practical lessons for engineers who run inference on TorchTPU.

1 Introduction

Engineers who serve language models on GPUs usually write PyTorch. Running the same code on TPUs has required a separate stack based on lazy tensors [1], where the programming model differs from the eager execution that PyTorch users know. TorchTPU [2] takes another approach. It registers a `tpu` device in PyTorch, so that `tensor.to("tpu")` behaves like `tensor.to("cuda")`: operations run eagerly, a compiler builds TPU programs in the background, and `torch.compile` [3] is available when more performance is needed.

A new backend raises practical questions. How fast is text generation today, which parts of the stack are ready, and what must a user write to get good performance? This report answers these questions for one workload. It measures greedy decoding of Gemma 2B [4] on one TPU v5e chip with the public release `torch_tpu` 0.2.0.

The workload is taken from a previous experiment by the same author. That experiment ran Gemma 2B under vLLM [5] with the JAX based TPU plugin [6] on a TPU v5e chip in the same hosted notebook service, and reached 19,511 tokens per second at batch 512 after tuning the block size of the attention kernel. Reusing the prompts, the batch size, the number of generated tokens, and the metric definitions

makes the two sets of numbers comparable at the level of the workload. Section 6.4 explains which parts of the two systems differ.

TorchTPU is a device backend, a peer of CUDA, and not a serving engine like vLLM. Everything that vLLM provides above the device had to be written: the model, the cache for attention keys and values, the decoding loop, and the attention kernel. The contributions of this report are:

1. A small inference harness for TorchTPU: a Gemma 2B written in plain PyTorch that loads the public checkpoint, a static key and value cache, and greedy decoding that stays on the device (Section 3).
2. A decode attention kernel written in Pallas [7] and registered as a PyTorch operation, verified on CPU against a float64 reference (Sections 3.4 and 4.4).
3. The use of the `scan` operator inside `torch.compile` to run several decode steps as one TPU program, which reduces the work done by the host for every token (Section 3.5).
4. Four controlled experiments that add one change at a time, from 2,095 to 21,645.8 tokens per second, with an analysis of where the remaining time goes (Sections 5 and 6).
5. A list of practical lessons for machine learning engineers and for TorchTPU and JAX developers (Section 7).

2 Background

2.1 Autoregressive decoding

A decoder language model produces text one token at a time. Generation has two phases. In the *prefill* phase the model reads the whole prompt in one forward pass and produces the first new token. In the *decode* phase it repeatedly takes the last token, runs one forward pass, and produces the next token. Each attention layer needs the keys and values of all previous positions. Recomputing them at every step would repeat work, so they are stored in a *key and value cache* (KV cache) and each decode step only computes the keys and values of the new token [5].

Greedy decoding picks the token with the highest score at every step. It needs no temperature, no random sampling, and no sorting, so it is the simplest selection rule to run on an accelerator. For a fixed model and fixed inputs it is deterministic, which makes it useful for verification.

Several sequences are usually decoded together as a *batch*. The model weights are read once per step for the whole batch, so a larger batch spreads the cost of reading the weights over more tokens. This is why throughput grows with batch size until the chip runs out of compute or memory.

2.2 TPU v5e and the roofline view

A TPU v5e chip has 16 GiB of high bandwidth memory (HBM), a memory bandwidth of 819 GB/s, and a peak of 197 TFLOP/s for bfloat16 matrix multiplication [8]. A program step takes at least the time to move its data through memory and at least the time to perform its arithmetic at peak rate. The ratio of arithmetic to memory traffic, called arithmetic intensity, decides which limit applies. This is the roofline model [9]. For TPU v5e the two limits meet at $197 \times 10^{12} / 819 \times 10^{9} \approx 240$ floating point operations per byte. A decode step with a small batch is below this value and is limited by memory bandwidth; a large batch can move above it and become limited by compute. Section 6.1 uses this model to explain the results.

2.3 TorchTPU execution modes

TorchTPU dispatches PyTorch operations to TPU programs compiled by XLA [10]. The release used here offers three ways to run code [11]:

- **Strict eager mode** is the default. Each operation becomes its own small compiled program, which is cached and executed asynchronously. Compilation is fast, but every operation pays a small launch cost on the host. When operations are short, the chip can wait for the host between them.
- **Defer and fuse mode** is optional. It keeps operations pending, detects repeated sequences such as the body of a loop, and compiles them together into larger programs.
- `torch.compile` captures a whole Python function as a graph with TorchDynamo [3], compiles it once into one TPU program, and runs that program on every later call. TorchTPU compiles with static shapes by default: each distinct tensor shape produces a separate program.

In all modes execution is asynchronous. The host queues work and continues; results become available when the host asks for them, for example by copying a tensor to the CPU or by calling `torch.accelerator.synchronize()`.

2.4 Pallas and the `jax_op` bridge

Pallas [7] is a kernel language inside JAX [12]. A Pallas kernel describes the work for one block of data and a grid of blocks; the compiler turns it into a TPU kernel and moves the blocks between HBM and the fast on chip memory (VMEM). TorchTPU provides `torch.tpu.pallas.jax_op`, which takes a JAX function, usually one that calls a Pallas kernel, and registers it as a PyTorch operation. The operation can be called from eager code and from code compiled with `torch.compile`. Arguments annotated as Python integers are treated as compile time constants, so each value creates its own kernel.

2.5 The reference experiment with vLLM

The reference experiment ran vLLM 0.23 with the TPU plugin on a TPU v5e chip. Gemma 2B was loaded from the public checkpoint and run with `max_model_len` 2048, up to 512 sequences, greedy decoding, and 128 generated tokens per sequence. The prompts were four short sentences repeated 128 times. With the stock attention kernel the run produced about 15,338 tokens per second. Copying the attention kernel source, overriding its key and value block size to 256, and patching the module references at runtime raised the result to 19,511 tokens per second, a gain of 27 percent. The same run reported a batch time to first token of 300 milliseconds, an inter token latency of 24.1 milliseconds, and an engine initialization of 173.9 seconds. This report reuses these definitions and numbers as its reference.

3 The inference stack written for TorchTPU

3.1 Model

The TorchTPU repository contains reference models, but none with the architecture of Gemma 2B, so the model was written in plain PyTorch. It follows the published architecture [4]: 18 decoder layers, hidden size 2048, an MLP of size 16384 with the tanh approximation of GELU [13], RMSNorm [14] computed in float32 with the scale $(1 + w)$, rotary position embeddings [15], and a vocabulary of 256,000 tokens. Attention is multi query attention [16]: 8 query heads share a single key and value head, each of dimension 256. The output layer reuses the embedding matrix, and embeddings are multiplied by $\sqrt{2048}$ rounded to bfloat16 (45.25), as in the reference implementation.

Attribute names match the Hugging Face checkpoint, so `google/gemma-2b` loads directly. The loader asserts that every tensor of the checkpoint is used; with real weights it reported 164 tensors, all consumed. Two choices differ from the checkpoint layout. The query, key, and value projections are concatenated into one matrix, and the gate and up projections of the MLP into another. One larger matrix multiplication replaces three or two smaller ones, and the outputs are split with views that do not copy memory.

3.2 Static shapes

With static shapes, every distinct tensor shape compiles a new program, and a guard check on a changing Python value also triggers a new compilation. The harness therefore keeps every shape and every Python value constant during generation:

- **Left padding.** Every prompt is padded on the left to 16 tokens. All sequences then sit at the same position at every step, so one position value drives the whole batch. A per sequence integer, `start`, records the first real token, and attention masks exclude the padded positions. Rotary embeddings depend only on the distance between positions, so the constant offset does not change the result.
- **Preallocated cache.** Each layer owns a key tensor and a value tensor of shape $(B, L, 256)$, where B is the batch size and $L = 256$ the maximum length. Prompts of up to 16 tokens plus 128 generated tokens need at most 143 positions. The single key and value head of multi query attention is stored without a head axis.
- **Position as a device tensor.** The current position is a tensor with zero dimensions stored on the TPU, not a Python integer. The compiled code reads it and increments it in place, so the compiler never sees a changing Python value. This follows the static cache pattern of the TorchTPU examples [11].
- **Masks and rotary angles computed inside the program.** The attention mask is computed from `start` and the position at every step, and the rotary angles from the position, instead of slicing precomputed tables. No tensor passed to the compiled function is a view with a moving offset.

3.3 Greedy decoding on the device

The decode step ends with `argmax` over the 256,000 scores, inside the compiled program. The chosen token is written to a preallocated output tensor of shape $(B, 128)$ at an index computed from the position, and becomes the input of the next step. No token travels to the host during generation. The host copies the output tensor once, after the last step. Listing 1 shows the function that `torch.compile` receives.

```
def decode_fn(start, pos, cur, out, ks, vs):
  nxt, nks, nvs = model.decode(cur, pos, start, ks, vs, inplace=True, **attn_kw)
  out.index_copy_(1, (pos - (P - 1)).view(1), nxt[:, None])  # store token
  cur.copy_(nxt)        # next input token
  pos.add_(1)           # zero dimensional device tensor, not a Python int
```

Listing 1: The decode step compiled with `torch.compile(dynamic=False, fullgraph=True)`. All state is updated in place on the device.

Inside `model.decode`, each layer writes the new key and value into the cache with `index_copy_` at the current position, then attends over the full cache with a mask. `fullgraph=True` makes compilation fail if anything forces a break in the graph, instead of silently splitting the step into several programs. Prefill is a second compiled function. Generation therefore uses two programs, one for prefill and one for

decode, and the harness counts the compiled graphs to show that no program is compiled again during measurement.

3.4 A Pallas decode attention kernel

During decode each sequence has one query token, which attends to up to 256 cached positions. Written with standard operations, this is two batched matrix multiplications with a mask and a softmax in between. The custom kernel computes the same result in one pass over the cache, in the style of FlashAttention [17] and flash decoding [18]. It processes the cache in blocks of `bkv` positions and keeps a running maximum, a running sum, and a running weighted sum of values in float32, which is the online softmax [19]. Scores and probabilities never leave on chip memory.

Figure 1 shows the grid. The first axis walks over groups of `bb` = 8 sequences and runs in parallel. The second axis walks over key and value blocks and carries the running statistics. The per sequence `start` values and the current position are small integers; they are passed through *scalar prefetch*, which places them in the scalar memory (SMEM) of the TPU core where the kernel and the block index functions can read them.

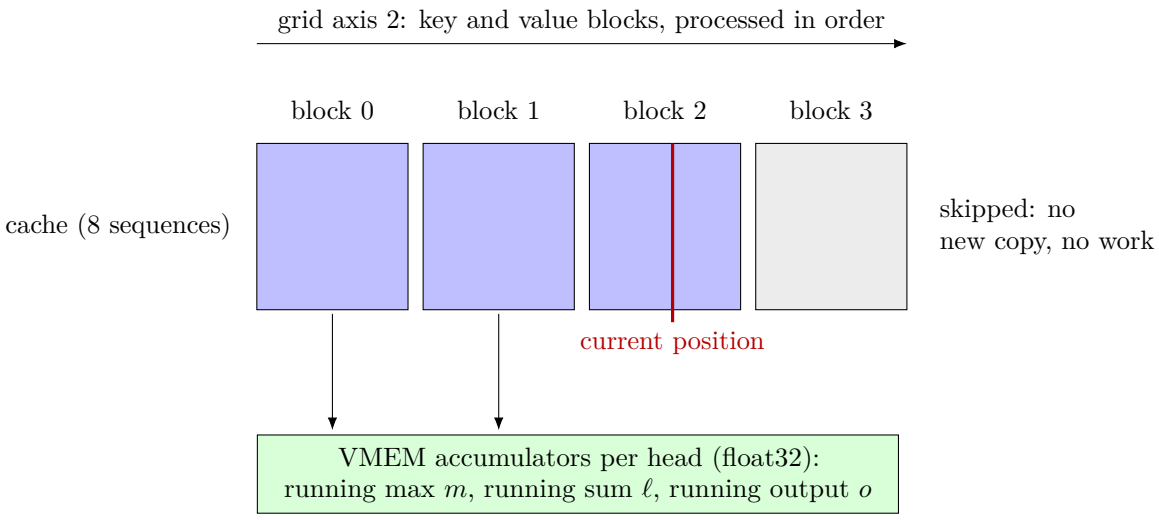


Figure 1: The decode kernel for one group of 8 sequences. Blocks of `bkv` cache positions are processed in order and update running softmax statistics. Blocks that lie entirely after the current position contain no valid keys; their block index is clamped to the last valid block, so no new data is copied and the kernel body does not run.

The kernel also skips work that cannot contribute. A block that starts after the current position contains no valid keys. The index function for the key and value blocks clamps such blocks to the last valid block index. When two consecutive grid steps request the same block, Pallas does not issue a new copy from HBM, and a guard in the kernel body skips the computation. Listing 2 shows the main parts.

```
def kv_map(b, j, pos_ref, start_ref):          # block index function
  return (b, jnp.minimum(j, pos_ref[0] // bkv), 0)   # clamp: skip dead blocks

@pl.when(j * bkv <= pos)                        # body runs only for live blocks
def _body():
  for i in range(bb):                           # 8 sequences per grid step
    s = dot(q_ref[i], k_ref[i].T) * scale        # (8 heads, bkv) in float32
    valid = (kv_idx >= start_ref[b * bb + i]) & (kv_idx <= pos)
    s = jnp.where(valid, s, NEG)
    m_new = maximum(m_prev, s.max(-1))           # online softmax update
    p = jnp.where(valid, exp(s - m_new), 0.0)
    alpha = exp(m_prev - m_new)
```

```
    l_new = alpha * l_prev + p.sum(-1)
    acc = alpha * acc + dot(p.astype(v.dtype), v_ref[i])

op = torch.tpu.pallas.jax_op("gemma_bench::decode_attention", _decode_attention)
out = op(q, k_cache, v_cache, start_i32, pos_i32, bkv, bb)  # bkv, bb: static ints
```

Listing 2: Core of the Pallas decode kernel and its registration as a PyTorch operation (simplified).

Unlike the runtime patching used in the vLLM experiment, the kernel is an ordinary PyTorch operation with the block size as an argument. A message printed when the operation is traced proves that the kernel is compiled into the program that runs.

3.5 Several decode steps in one program

Even with `torch.compile`, the host does work at every token. Python runs one loop iteration, TorchDynamo checks the guards of the compiled function, the runtime prepares about 40 arguments (two cache tensors per layer plus the state), and the program is queued. GPU inference meets the same cost as kernel launch overhead and reduces it with CUDA Graphs, which record a sequence of launches and replay it as one unit [20].

PyTorch includes a higher order operator, `torch.ops.higher_order.scan`, which applies a function repeatedly and carries a state from one iteration to the next. Under `torch.compile`, TorchTPU lowers it to a loop that runs on the TPU. The harness uses it to run K decode steps per call (Listing 3). The carried state contains the current token, the position, and the 36 cache tensors; the per step output is the generated token. The cache is updated functionally inside the loop. For 127 decode steps and $K = 8$, generation runs 15 calls of the loop program and 7 single steps, instead of 127 single steps. Figure 2 compares the two schedules.

```
from torch._higher_order_ops import scan

def scan_fn(start, pos, cur, out, ks, vs):
  def body(carry, _x):
    c_cur, c_pos = carry[0], carry[1]
    nxt, nks, nvs = model.decode(c_cur, c_pos, start, carry[2:2+n],
                                 carry[2+n:], inplace=False, **attn_kw)
    return [nxt, c_pos + 1, *nks, *nvs], nxt.clone()
  carry, ys = scan(body, [cur, pos, *ks, *vs], torch.zeros(K, device=start.device))
  out.index_copy_(1, (pos - (P - 1)) + torch.arange(K, device=start.device),
                  ys.transpose(0, 1).contiguous())
  cur.copy_(carry[0]); pos.copy_(carry[1])
  return list(carry[2:2+n]), list(carry[2+n:])   # new caches, rebound by the host
```

Listing 3: Running K greedy decode steps as one TPU program with `scan`.

3.6 Measurement harness

A single script runs one configuration and prints a block of metrics. Configuration is passed through environment variables (Appendix A). Each run is a separate operating system process, because the TPU runtime allows one process per chip; the notebook that drives the runs never initializes the TPU itself. A run has four phases:

1. **Build.** Create the model on the TPU and load random or real weights. Random weights have the same shapes and the same amount of work as the real ones, so they give valid throughput but meaningless text.
2. **Warmup.** Generate a short sequence with the full batch. This compiles the prefill program, the decode program and, when enabled, the loop program.

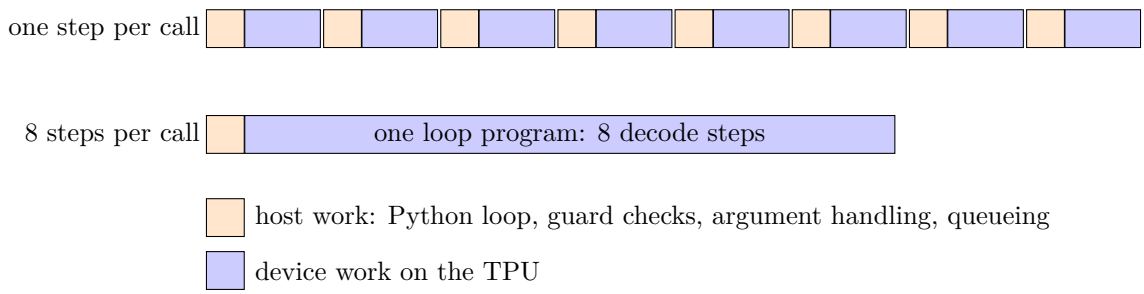


Figure 2: Host and device work per decode step. With one step per call, the host repeats its fixed work for every token. With `scan`, the host work happens once for K steps. Execution is asynchronous, so host work overlaps with device work only if the host finishes its part before the device finishes the previous program.

3. **Batch time to first token.** Run prefill for the whole batch, produce the first token, and wait for the device.
4. **Main run.** Generate 128 tokens per sequence, copy the tokens to the host, and stop the clock.

The host waits for the device after every 8 decode steps, which keeps the queue of pending programs bounded, and once at the end.

4 Methodology

4.1 Workload and setup

Table 1 lists the setup. The prompts are those of the reference experiment: *Chirality is*, *The birthday of Al Pacino is*, *PyTorch is*, and *Brazil celebrate its independence on the days*. They are repeated to form batches of 128 or 512 sequences.

Table 1: Experimental setup.

Item	Value
Hardware	one TPU v5e chip (16 GiB HBM), hosted notebook runtime
TorchTPU	`torch_tpu` 0.2.0
PyTorch	2.11.0 (CPU build, as required by TorchTPU)
JAX, jaxlib	0.9.2 (Pallas kernel)
libtpu	0.0.50
Model	Gemma 2B, bfloat16 weights; random or `google/gemma-2b`
Prompt length	left padded to 16 tokens
Generated tokens	128 per sequence (1 from prefill, 127 decode steps)
Cache length	256 positions, dense, one tensor pair per layer
Decoding	greedy (argmax), no sampling
Batch sizes	128 (experiment 1), 512 (experiments 2 to 4)
Compilation	`torch.compile(dynamic=False, fullgraph=True)`, strict eager mode around it
Compilation cache	persistent cache shared across runs

4.2 Metrics

The metrics follow the reference experiment, with B sequences and $N = 128$ generated tokens per sequence:

- **Throughput**: BN/T_{gen}, where T_{gen} is the wall clock time of the main run, including prefill and the final copy of the tokens to the host.

- **Batch time to first token**: T_{ttft}, the wall clock time of prefill plus the first token for the whole batch. It is a batch measure, not the latency of one request.
- **Inter token latency**: $(T_{\text{gen}} - T_{\text{ttft}})/(N-1)$. Because all sequences advance together, this is also the time of one decode step for the whole batch.
- **Warmup time**: the wall clock time of the warmup phase, which is dominated by compilation.

Energy is not reported. The environment exposes no power telemetry, and no measured or published power figure for the chip was available.

4.3 Rules for valid measurements

Four rules were applied to every run.

1. **One change per experiment.** Each experiment changes one setting relative to the best configuration found so far, and the next experiment starts from the winner.
2. **No silent CPU execution.** TorchTPU can fall back to the CPU for operations it does not implement. The fallback was disabled, so a missing TPU operation would stop the run instead of producing a number that is not a TPU measurement.
3. **Evidence printed by every run.** Each run prints the execution mode, the fallback state, that gradients are disabled and no parameter requires gradients, the device, the number of compiled graphs after warmup and after the main run, and a checksum of the generated tokens. For the Pallas kernel it prints the trace message and the maximum difference between the kernel and the standard attention on random inputs, computed on the TPU.
4. **No quotes from perturbed runs.** Runs with profiling, with a synchronization after every step, or with the debug execution mode are marked as not quotable, and the analysis code excludes them. None of the runs reported here are of that kind.

4.4 Verification on CPU before running on the TPU

Two checks ran on the CPU before any TPU run (Table 2). The model check uses a small configuration in float32: the logits of cached prefill and decode match an uncached forward pass over the same sequence; the fused projection layout gives the same greedy tokens as separate projections; and functional cache updates give the same tokens as in place updates. The kernel check runs the Pallas kernel in the Pallas interpreter on the CPU and compares it with a float64 reference, for positions 15, 40, 143, and 255, left padding of up to 15 positions, and each block size. The results differ slightly between block sizes, because a different split into blocks changes the order of floating point additions in the online softmax; the spread is below 3×10^{-7} in float32.

Table 2: CPU verification before the TPU runs.

Check	Result
Cached decode vs uncached forward, max logit difference (float32)	7.7×10^{-6}
Fused vs separate projections, greedy tokens	identical
In place vs functional cache updates, greedy tokens	identical
Kernel vs float64 reference, float32 inputs, worst case over all tests	1.2×10^{-6}
Kernel vs float64 reference, bfloat16 inputs, worst case over all tests	7.7×10^{-3}
Kernel spread across block sizes 64, 128, 256 (float32)	$\leq 2.7 \times 10^{-7}$

5 Results

Table 3 lists all nine runs. Every run passed the evidence checks: strict eager mode, CPU fallback disabled, gradients disabled, and the same number of compiled graphs after warmup and after the main run, which means that nothing was compiled during measurement.

Table 3: All runs. Throughput in tokens per second, latencies in milliseconds, warmup in seconds. "XLA" is the attention written with standard PyTorch operations and compiled by XLA; "Pallas" is the custom kernel. All runs use `torch.compile` except the first.

Exp.	Configuration	Batch	Throughput	ITL	TTFT	Warmup	Peak HBM (GiB)
1	strict eager, no compile, XLA	128	2,095.0	60.7	108	2.5	6.7
1	`torch.compile`, XLA	128	10,624.0	11.7	54	17.1	11.3
2	XLA attention	512	16,923.2	28.4	263	19.9	15.7
2	Pallas, `bkv` 64	512	17,283.4	28.1	228	10.6	15.7
2	Pallas, `bkv` 128	512	17,620.4	27.5	227	10.6	15.7
2	Pallas, `bkv` 256	512	17,872.0	27.1	229	10.7	15.7
3	Pallas 256, scan $K = 8$	512	21,639.0	22.1	222	19.8	15.7
3	Pallas 256, scan $K = 32$	512	21,320.5	22.5	222	17.2	15.7
4	Pallas 256, scan 8, real weights	512	**21,645.8**	22.1	222	11.4	15.7

5.1 Experiment 1: eager execution versus `torch.compile`

At batch 128 with the default strict eager mode, the model generates 2,095 tokens per second, with a decode step of 60.7 milliseconds. Compiling the decode step with `torch.compile` raises throughput to 10,624 tokens per second and reduces the step to 11.7 milliseconds, a factor of 5.1 (Figure 3a). Both runs produced the same token checksum.

The cost is compilation: warmup takes 2.5 seconds in eager mode, where each operation is a small program, and 17.1 seconds with `torch.compile`. Peak memory grows from 6.7 to 11.3 GiB.

The size of the gap follows from the structure of the step. One decode step of this model contains several hundred operations: 18 layers, each with projections, normalizations, rotary embeddings, cache writes, the mask, the softmax, and the MLP. In strict eager mode each operation is a separate program with its own launch on the host. Many of these operations are small at batch 128, so the chip waits for the host between them. A compiled step is one program, launched once, which the compiler can also optimize as a whole, for example by fusing element wise operations into the matrix multiplications around them.

5.2 Experiment 2: the Pallas decode kernel

At batch 512, the compiled model with standard attention generates 16,923 tokens per second. Moving from batch 128 to 512 raises throughput by a factor of 1.6, while the step time grows from 11.7 to 28.4 milliseconds. Replacing the attention with the Pallas kernel raises throughput at every block size (Figure 3b): by 2.1 percent with `bkv` 64, 4.1 percent with 128, and 5.6 percent with 256, which reaches 17,872 tokens per second. All four runs produced the same token checksum, so the kernel changes the speed and not the generated text in this test. The difference between kernel and standard attention on random inputs is 0.012 in bfloat16, which is in the range of bfloat16 rounding for values of this size.

The best block size is 256, the full cache length. With one block per sequence group, the kernel never skips a block. The smaller block sizes skip on average about half of the cache, because the position stays below 143, but they need two to four times as many grid steps. The measured order shows that, at this

cache length, the number of grid steps costs more than the skipped memory traffic saves. Section 6.3 discusses this further.

Warmup is shorter with the kernel, 10.6 against 19.9 seconds, but all runs share a persistent compilation cache, so these are not cold compilation times. The batch time to first token was 263 milliseconds in the XLA run and 227 to 229 milliseconds in the kernel runs. Prefill uses the same code in all four runs, so with one run per configuration this difference is treated as variation between processes.

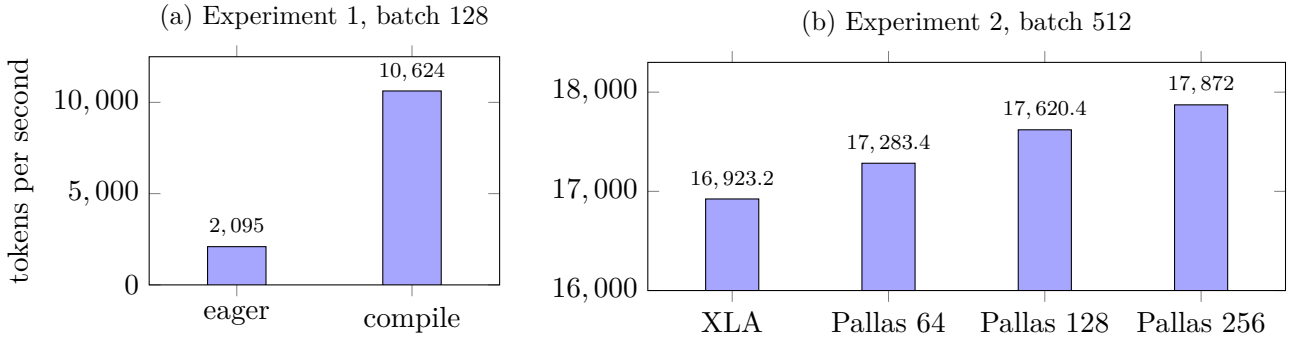


Figure 3: (a) Compiling the decode step raises throughput by a factor of 5.1. (b) The Pallas kernel at three block sizes against the attention compiled by XLA; the vertical axis starts at 16,000 tokens per second to make the differences visible.

5.3 Experiment 3: several decode steps per program

Running eight decode steps per call with `scan` raises throughput from 17,872 to 21,639 tokens per second, a gain of 21.1 percent, and reduces the step time from 27.1 to 22.1 milliseconds. The tokens are again identical. The loop program adds a third compiled graph, and warmup grows to 19.8 seconds.

With 32 steps per call, throughput is 21,321 tokens per second, 1.5 percent lower than with 8. The likely reason is the schedule: 127 decode steps split into 3 calls of 32 steps plus 31 single steps, while with 8 steps per call only 7 steps remain outside the loop. With $K = 32$ a quarter of the tokens are still produced with one call per step.

5.4 Experiment 4: real weights

The last run uses the configuration of experiment 3 with $K = 8$ and loads the `google/gemma-2b` checkpoint. It generates 21,645.8 tokens per second, within 0.03 percent of the random weight run. Same shapes give the same work, so this confirms that random weights give valid throughput numbers. The main generation was repeated in the same process and produced identical tokens, which checks that greedy decoding is deterministic on this stack.

Model build, including reading the checkpoint, took 45.9 seconds and warmup 11.4 seconds, for 57.3 seconds before the first measured generation. The reference vLLM run reported an engine initialization of 173.9 seconds; vLLM also compiles programs for many shapes that this harness does not need, so the two numbers measure different amounts of work.

The outputs are grammatical sentences on the topic of each prompt. The beginnings of the outputs for the four prompts were:

- *Chirality is* a fundamental property of chiral molecules, which is defined as the difference between the number of ...
- *The birthday of Al Pacino is* on 25th May. He is an American actor, director, producer, and screenwriter. ...

- *PyTorch is* a popular deep learning framework that is used by many researchers and developers. . . .
- *Brazil celebrate its independence on the days* of 7th and 12th of September. . . .

Some statements are wrong: Al Pacino was born on 25 April, and the second date for Brazil is not correct. This is the behavior of a small base model without instruction tuning, not of the inference stack; the outputs are shown to confirm that the weights are loaded correctly and that decoding produces coherent text.

5.5 Summary of the gains

Figure 4 shows the configurations at batch 512 next to the two reference numbers from the vLLM experiment. Relative to the compiled model with standard attention, the kernel adds 5.6 percent and the loop program adds 21.1 percent, for a total gain of 27.9 percent. The final number is 10.9 percent above the tuned vLLM result and 41 percent above the vLLM result with the stock kernel.

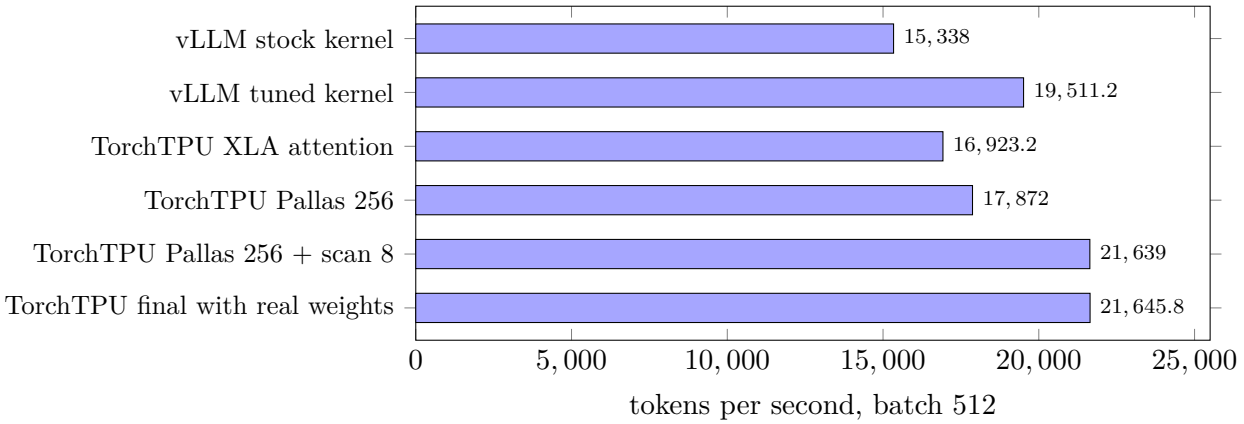


Figure 4: Throughput at batch 512. The vLLM numbers come from the reference experiment on the same chip class, prompts, and token count; Section 6.4 lists the differences between the two systems.

6 Analysis

6.1 Where the time goes

A count of the work per step explains the shape of the results. Gemma 2B has about 2.51 billion parameters, of which 0.52 billion are in the embedding matrix. Generating one token takes about 5.0 billion floating point operations in matrix multiplications, two per weight, including the output layer that reuses the embedding matrix. Attention over a 256 position cache adds about 0.04 billion. Table 4 applies the roofline model to the measured step times. The memory traffic counts the weights, the full key and value cache, and the logits; it ignores activations and assumes that nothing is read twice. These are estimates, not profiler measurements.

At batch 128 the step is below the ridge point of about 240 operations per byte, so it is limited by memory; the measured 11.7 milliseconds is 1.7 times the memory bound. At batch 512 the arithmetic intensity is 336 operations per byte, above the ridge point, so the step is limited by compute: 13.1 milliseconds at peak against 22.1 measured. The final configuration therefore uses about 55 percent of the peak bfloat16 compute of the chip. Two consequences follow. First, attention is a small part of the arithmetic at this cache length, which limits what any attention kernel can gain; the kernel gained 5.6 percent. Second, a larger batch would raise the arithmetic per step in proportion, so throughput would grow slowly if at all, and the peak memory of 15.7 GiB already leaves little space.

Table 4: Estimated lower bounds for one decode step against the measured step time. Achieved compute is throughput times the operations per token.

	Batch 128	Batch 512
Arithmetic per step (TFLOP)	0.65	2.59
Memory traffic per step (GB)	5.68	7.69
Arithmetic intensity (FLOP per byte)	114	336
Time at peak compute (ms)	3.3	13.1
Time at peak memory bandwidth (ms)	6.9	9.4
Limiting resource	memory	compute
Measured step time (ms)	11.7	22.1
Achieved compute (TFLOP/s)	53.7	109.3
Fraction of peak compute	27%	55%

6.2 Why the loop program helps

Experiment 3 reduced the step time by 5.0 milliseconds without changing the arithmetic on the device. That time was spent outside the TPU program: in the Python loop, in the guard checks of TorchDynamo, in preparing about 40 arguments for every call, and in queueing the program. With asynchronous execution this cost would be hidden if the host always finished first; the measurement shows that it was not, either because the host was slower for part of each step or because the waits after every 8 steps exposed it. Running 8 steps per call removes seven eighths of this work. After that, 22.1 milliseconds remain, of which the compute bound accounts for 13.1. The rest is likely spent in element wise operations, normalizations, the 256,000 way `argmax`, the cache writes inside the loop, and the parts of the matrix multiplications that do not reach peak. A device profile would separate these terms.

6.3 Why the largest block size is the fastest

Block skipping saves memory traffic for blocks after the current position. With `bkv` 64 the kernel reads one block of 64 positions for positions below 64, two blocks up to 127, and three blocks up to 143, so on average about half of the 256 positions. With `bkv` 256 it always reads all 256. Yet the largest block was the fastest. Two effects explain this. Each grid step has a fixed cost on the TPU core, and the number of steps per layer is $\frac{512}{8} \times \frac{256}{\mathrm{bkv}}$, which is 256 steps for `bkv` 64 and 64 for 256. And at batch 512 the step is limited by compute, so saving memory traffic in attention has little effect on the total. At longer cache lengths, where attention reads much more data and the position is far from the end of the cache for most of the generation, the balance would change.

6.4 Comparison with the vLLM experiment

The TorchTPU result is 10.9 percent above the tuned vLLM result. Table 5 lists what the two runs share and where they differ. The comparison shows that the TorchTPU stack can reach this throughput for this workload. It does not show that one system is faster in general.

The main differences favor the TorchTPU harness. It does no scheduling and no detokenization, it knows the maximum length of every sequence in advance, and its cache holds 256 positions instead of 2048. vLLM supports requests that arrive and leave at any time and have any length, which a fixed batch with left padding does not. In the other direction, left padding makes the harness process padded positions that vLLM's ragged batching skips; with prompts of at most 16 tokens this cost is small. A fair summary is that, for a fixed batch of short prompts, a PyTorch program on TorchTPU with one custom kernel and one loop program reaches the same range of throughput as a tuned serving engine on the same chip.

Table 5: The reference vLLM run and the final TorchTPU run.

	vLLM with TPU plugin	TorchTPU harness
Chip	TPU v5e, one chip	TPU v5e, one chip
Model and weights	Gemma 2B, real	Gemma 2B, real
Prompts, batch, tokens	same	same
Decoding	greedy	greedy
Maximum length	2048	256
Cache	paged, ragged batching	dense, left padded
Scheduling	continuous batching, request queue	fixed batch, no queue
Output processing	detokenized text per request	token ids, one copy at the end
Attention kernel	ragged paged attention, block 256	custom decode kernel, block 256
Throughput (tok/s)	19,511.2	21,645.8
Batch time to first token (ms)	300	222
Inter token latency (ms)	24.1	22.1
Initialization (s)	173.9	57.3

7 Practical lessons

The following points apply to anyone who writes inference code for TorchTPU, and several apply to `torch.compile` in general.

1. **Compile the decode step.** In strict eager mode each operation is its own program, and the host launch cost dominates when operations are small. Compiling the whole step was the largest single gain (5.1 times).
2. **Keep every changing value on the device.** A Python integer that changes at every step becomes a guard and causes a new compilation. Store the position as a tensor with zero dimensions, update it in place inside the compiled function, and compute masks and rotary angles from it inside the program.
3. **Allocate state inside `torch.inference_mode()`.** During development, in a dry run of the harness on the CPU, the decode program was compiled again in the middle of a measurement. The cache tensors had been allocated outside inference mode, while the tensors returned by the compiled function were created inside it. Tensors of the two kinds carry different dispatch keys, TorchDynamo guards on them, and the first call with a returned tensor failed the guard. Allocating all state inside inference mode fixed it. The symptom is a jump in the number of compiled graphs during measurement, which is why the harness prints that number.
4. **Use `fullgraph=True` and count the graphs.** A break in the graph splits the step into several programs and adds host work silently. With `fullgraph=True` the compilation fails instead. Comparing the number of compiled graphs after warmup and after the main run proves that nothing was compiled while measuring.
5. **Warm up every program that will run.** The loop program only runs when at least K steps remain. A warmup that is shorter than K steps would leave its compilation inside the measured region. The harness warms up with at least $K + 2$ steps.
6. **Choose K to cover the whole generation.** The steps that remain after the last full loop run one at a time. With 127 decode steps, $K = 8$ leaves 7 single steps and $K = 32$ leaves 31, which made the larger value slower.
7. **Register kernels with `jax_op` instead of patching.** A Pallas kernel registered as a PyTorch operation works in eager and compiled code, takes its tuning parameters as arguments, and needs

no runtime replacement of module references. A message printed at trace time shows that the kernel is in the program that runs.

8. **Pass per sequence integers through scalar prefetch.** Small integer arrays such as the start of each sequence and the current position belong in scalar memory. Giving them a blocked layout violates the TPU tiling rules, which require the last two dimensions of a block to be multiples of 8 and 128 or equal to the full array.
9. **Verify kernels on the CPU first.** The Pallas interpreter runs TPU kernels on the CPU. Comparing them with a float64 reference catches indexing and masking errors before any TPU time is used. Expect small differences between block sizes, since the order of additions changes.
10. **Use one process per run.** The TPU runtime allows one process per chip. Running each configuration as a separate process from a notebook that never touches the TPU avoids conflicts and makes each measurement independent.
11. **Random weights are enough for throughput.** With the same shapes, random weights gave the same throughput as the real checkpoint within 0.03 percent, which saves the download and loading time while tuning.

8 Limitations

- **One run per configuration.** Each number comes from one process on one chip in a hosted environment. Differences of a few percent, such as between block sizes 128 and 256, are within the range that could change between sessions.
- **Narrow workload.** Short prompts, a cache of 256 positions, a fixed batch, greedy decoding, and one 2 billion parameter model. Longer contexts make attention a larger share of the work and change the best kernel settings.
- **Not a serving system.** The harness has no request queue, no continuous batching, no paged cache, and no detokenization. Its numbers describe batch generation, not a service under load.
- **No device profile.** The analysis of where time is spent relies on counting work and on wall clock measurements. Warmup times were measured with a shared persistent compilation cache.
- **No energy figures.** The environment exposes no power telemetry.
- **One software version.** All results describe `torch_tpu` 0.2.0 with the versions in Table 1. TorchTPU is under active development, and the results should not be projected onto later releases.

9 Conclusion

This report measured greedy decoding of Gemma 2B on one TPU v5e chip with the public TorchTPU release 0.2.0. A model written in plain PyTorch, with a static cache and greedy decoding kept on the device, reached 10,624 tokens per second at batch 128 once the decode step was compiled, 5.1 times the eager result. At batch 512 a Pallas decode kernel registered with `jax_op` added 5.6 percent, and running eight decode steps per program with `scan` added 21.1 percent. With the real checkpoint the final configuration generated 21,645.8 tokens per second, with identical tokens across repeated runs, a batch time to first token of 222 milliseconds, and an inter token latency of 22.1 milliseconds. On the same workload, a tuned vLLM configuration on the same chip class had reached 19,511 tokens per second, with differences between the two systems that Section 6.4 lists. An estimate of the arithmetic places the final configuration at about 55 percent of the chip's peak bfloat16 compute.

Three directions follow from the analysis. A device profile would show how the remaining 9 milliseconds between the compute bound and the measured step are spent. Longer contexts would test the block skipping of the kernel where attention reads more data. And a scheduler with a paged cache on top of this harness would allow a comparison with vLLM as a serving system rather than as a batch generator.

Code availability

The code used in this study (model, kernel, harness, and the notebook that ran the experiments) is available from the author on request.

Acknowledgments

The author thanks the TorchTPU team for the public release, the documentation, and the examples that this work builds on.

References

[1] Alex Suhan, Davide Libenzi, Ailing Zhang, Parker Schuh, Brennan Saeta, Jie Young Sohn, and Denys Shabalin. LazyTensor: combining eager execution with domain specific compilers. *arXiv preprint arXiv:2102.13267*, 2021.

[2] Google. TorchTPU: A PyTorch backend for Google TPUs. https://github.com/google-pytorch/torch_tpu, 2026. Release 0.2.0, https://pypi.org/project/torch-tpu/0.2.0/.

[3] Jason Ansel et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[4] Gemma Team. Gemma: Open models based on Gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[6] vLLM Project. tpu-inference: TPU support for vLLM. https://github.com/vllm-project/tpu-inference, 2025.

[7] JAX Authors. Pallas: a JAX kernel language. https://docs.jax.dev/en/latest/pallas/index.html, 2024.

[8] Google Cloud. TPU v5e. https://cloud.google.com/tpu/docs/v5e. Accessed October 2026.

[9] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[10] OpenXLA Project. XLA: an open source machine learning compiler. https://openxla.org/xla, 2024.

[11] Google. TorchTPU documentation: execution model, custom kernels, and the Muse Glimmer inference case study. https://torchtpu.readthedocs.io/, 2026.

[12] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs. http://github.com/jax-ml/jax, 2018.

[13] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

[14] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[15] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[16] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[17] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[18] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-Decoding for long-context inference. https://crfm.stanford.edu/2023/10/12/flashdecoding.html, 2023.

[19] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.

[20] NVIDIA. CUDA graphs, CUDA C++ programming guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs, 2024.

A Harness configuration and run evidence

All configuration of the harness is passed through environment variables (Table 6).

Table 6: Environment variables of the measurement harness, with defaults.

Variable	Default	Meaning	Variable	Default	Meaning
`BATCH_MULT`	8	batch is 4 times this	`RPA_BKV`	128	kernel block size
`MODEL`	random	or `google/gemma-2b`	`PALLAS_BB`	8	sequences per grid step
`MAX_LEN`	256	cache length	`SYNC_EVERY`	8	steps between waits
`PROMPT_PAD`	16	left padded prompt length	`INPLACE_KV`	1	in place cache updates
`MAX_TOKENS`	128	tokens per sequence	`FUSED_QKV`	1	fused projections
`COMPILE`	1	use `torch.compile`	`DECODE_SCAN`	0	steps per loop program
`EXEC_MODE`	strict	strict, fuse, or debug	`DETERMINISM`	0	repeat and compare tokens
`CPU_FALLBACK`	0	allow CPU fallback	`STEP_TIMING`	0	wait after every step
`ATTN`	xla	xla or pallas	`PROFILE`	0	profiler trace

Run order and evidence. The nine runs were executed in the order of Table 3, each as a separate process, and all nine printed the expected evidence: execution mode `EagerMode.DEFER_NEVER` (strict) with CPU fallback `False`; device `TPU v5e` with 16.00 GiB of HBM; gradients disabled; 0 compiled graphs for the eager run, 2 for the compiled runs without the loop program and 3 with it, equal after warmup and after the main run. The token checksums were `29ca5e69363a` for both runs of experiment 1, `ef58b9f6c2b1` for all random weight runs at batch 512, and `a49fbe084f60` for the real weight run. The kernel runs printed the trace message with shapes $(512, 8, 256)$ for queries and $(512, 256, 256)$ for the cache, and a maximum difference of 0.0117 against the standard attention on random bfloat16 inputs. The real weight run loaded 164 checkpoint tensors, all consumed, and produced identical tokens in two repeated generations.